

蓝桥杯30天算法冲刺集训



Day-21 伸展树 Splay

伸展树 Splay

定义

首先，伸展树(splay tree)是一颗二叉搜索树，它的定义是建立在二叉搜索树之上，并且它是基于类似程序**局部性原理**的假设：**一个结点在一次被访问后，这个结点很可能不久再次被访问**。为了使整个查找时间更小，被查频率高的那些节点应当经常处于靠近树根的位置。这样，很容易得想到以下这个方案：每次对伸展树进行操作后，它均会通过旋转的方法把被访问节点旋转到树根的位置，这种自调整形式的二叉查找树就是伸展树。这个旋转操作被称作**伸展操作**。

正像程序局部性原理的实际效率被广泛证明一样，伸展树在实际的搜索效率上也是非常高效的。尽管存在最坏情况下单次操作会花费 $O(N)$ 的时间，但是这种情况并不是经常发生，而实际证明伸展树能够保证 M 次连续操作最多花费 $O(M \log N)$ 的时间。

相比于平衡二叉树，伸展树有差不多的平均性能，其他的优势在于：不需要存储平衡信息。另外如果采用自顶向下的调整方式，还能省去额外的递归栈开销。

实现

存储结构

一般我们把树上的每一个节点都作为结构体存放，节点中维护当前节点对应的值，当前节点对应值的个数，两个子节点的编号，子树中节点的个数。相比于普通平衡树，在 Splay 中，我们还要多存储父节点编号信息。

```
struct Node {
    Node *ch[2], *fa; // 左右孩子指针和父指针
    int val, cnt, siz;
    Node(int val): val(val), cnt(1), siz(1) {
        ch[0] = ch[1] = fa = nullptr;
    }
};
```

```

    }
    Node &lc() { return son[0]; } // &函数：在内存中不产生被返回值的副本
    Node &rc() { return son[1]; } // 用这种方法来封装取子结点
    bool is_rson() { // 判断当前结点是不是右孩子，用于 splay 操作
        if(fa==nullptr) return false;
        return fa->son[1] == this;
    }
    void update_siz() { // 更新结点信息
        siz = cnt;
        if(son[0] != nullptr) siz += son[0]->siz;
        if(son[1] != nullptr) siz += son[1]->siz;
    }
};

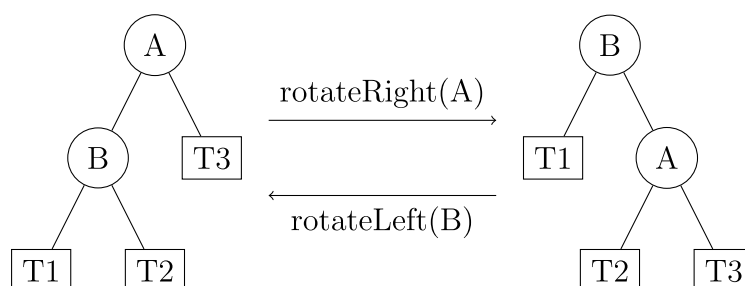
class Splay{
public:
    Node *root = nullptr;
    ...
};

```

旋转

为了使 Splay 保持平衡而进行旋转操作，旋转的本质是将某个节点上移一个位置。

旋转可以说是 Splay 的基础，利用旋转才能让树保持平衡，Splay 中有左旋（Left Rotation）和右旋（Right Rotation）。这个在我们学习 Treap 的时候已经做过讲解，这里不再赘述原理。



旋转需要保证：

- 整棵 Splay 的中序遍历不变（不能破坏二叉查找树的性质）。
- 受影响的节点维护的信息依然正确有效。

- `root` 必须指向旋转后的根节点。

因为我们并不在意到底什么时候需要左旋什么时候需要右旋，只要每次旋转都会将被旋转的节点的深度减少 1 就能满足我们的需求，并且，我们已经存储了每个结点的父结点地址（下标）。所以，在具体实现中，旋转操作和 Treap 中的实现细节略有不同，我们可以将左旋和右旋通过同一个函数实现，传参时也无需传入旋转类型的参数。。

具体分析旋转步骤（假设需要旋转的节点为 x ，其父亲为 y ，以右旋为例）

1. 将 y 的左儿子指向 x 的右儿子，且 x 的右儿子（如果 x 有右儿子的话）的父亲指向 y ；
2. 将 x 的右儿子指向 y ，且 y 的父亲指向 x ；
3. 如果原来的 y 还有父亲 z ，那么把 z 的某个儿子（原来 y 所在的儿子位置）指向 x ，且 x 的父亲指向 z 。

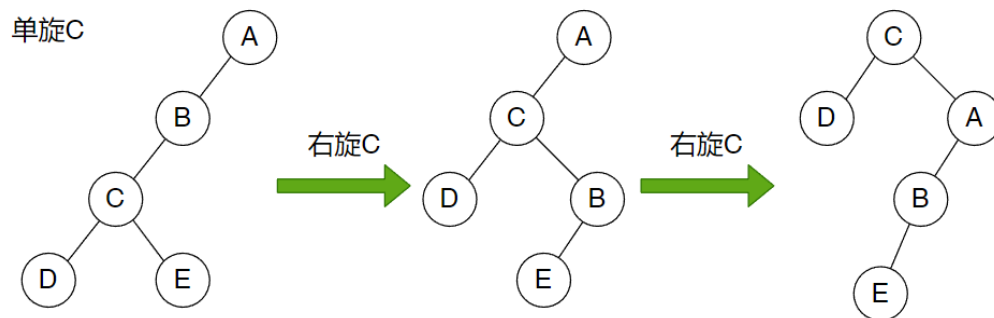
```
// 指针实现
void rotate(Node *x) {
    // y: 父结点   k: x 是否是右孩子
    Node *y = x->fa;
    int k = x->is_rson(); // 当前结点 x 在父结点的 ch[] 中的下标
    // x 上旋（左旋 or 右旋）
    y->ch[k] = x->ch[k^1];
    if(x->ch[k^1] != nullptr)
        x->ch[k^1]->fa = y; // 将 x 的反向孩子链向 y
    if((x->fa = y->fa) != nullptr) // 将 x 上旋至 y 的位置
        x->fa->ch[y->is_rson()] = x;
    y->fa = x; // y 下旋成 x 子结点
    x->ch[k^1] = y;
    x->update_siz();
    y->update_siz();
    // 更新 root
    if(x->fa == nullptr) root = x;
}
```

Splay操作

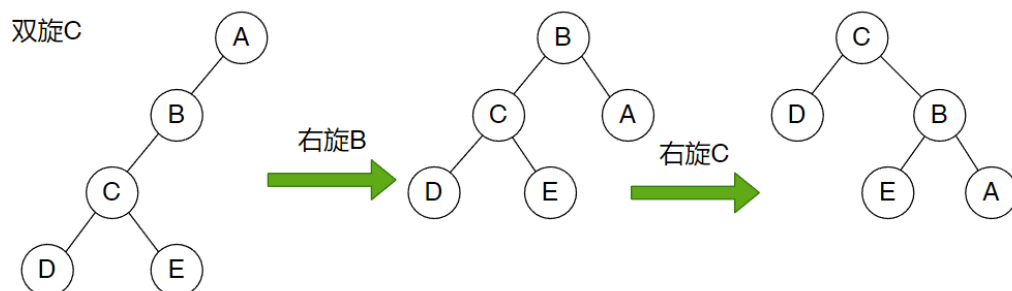
Splay 操作是 Spaly 树的核心操作，即为把一个节点通过不断旋转旋转到根。每次查询或者修改操作后都将操作的节点 Splay 到根就能保证单次操作时间复杂度均摊为 $\log n$ 。

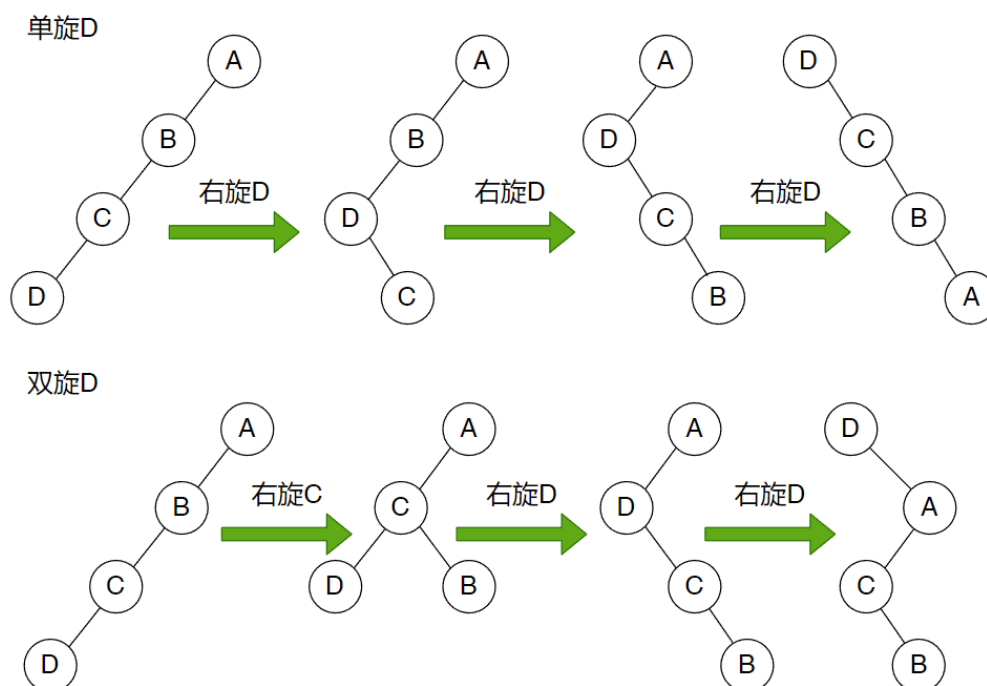
单旋 VS 双旋

但是我们不能单纯地把节点不断向上转，考虑如下这种情况，我们想把 C 转到根。



我们发现如果单纯把C向上转的话，如果原本是链，转动之后还是链，并不能优化，所以我们需要**双旋**，也就是在当前节点和当前节点的父节点都是自己父节点的左儿子或者右儿子时，我们先转动当前节点的父节点，再转动当前节点，如下图：





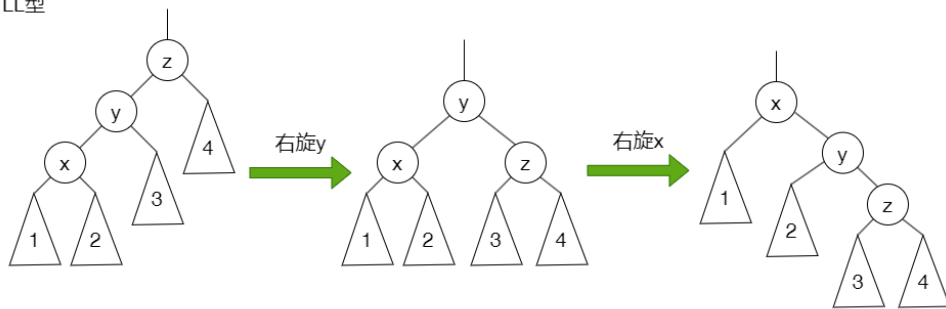
注意：*splay*不要写单旋，单旋的时间复杂度无法保证。单旋是*splay* 初学者的常见错误

如何旋转到树根

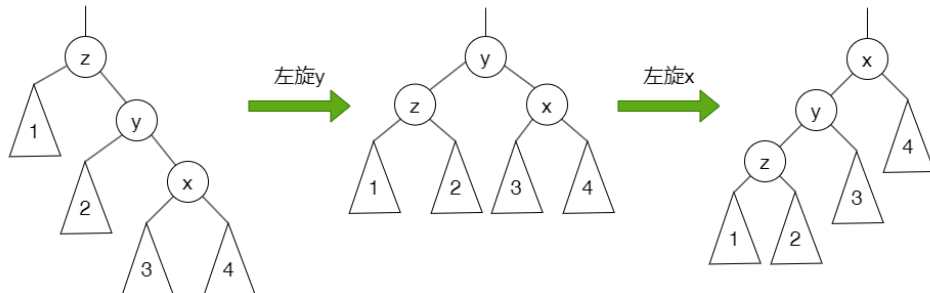
那么，一个结点如何通过双旋，Splay 到树根呢？这就要分类讨论了。这里，我们假设被操作的结点是 x ，其父结点是 y ，父结点的父结点是 z 。那么，可以将所有的可能分成五种情况：

1. LL 型： x 和 y 都是左孩子，此时需要先右旋 x 的父亲 y ，再右旋 x 本身。
2. RR 型： x 和 y 都是右孩子，此时需要先左旋 x 的父亲 y ，再左旋 x 本身。
3. LR 型： x 是 y 的左孩子，但是 y 是 z 的右孩子，此时需要先右旋 x ，再左旋 x 。
4. RL 型： x 是 y 的右孩子，但是 y 是 z 的左孩子，此时需要先左旋 x ，再右旋 x 。
5. 单 L 或单 R 型： x 是 y 的左孩子或者右孩子，并且 y 是树根，此时只需要右旋或者左旋 x 即可。

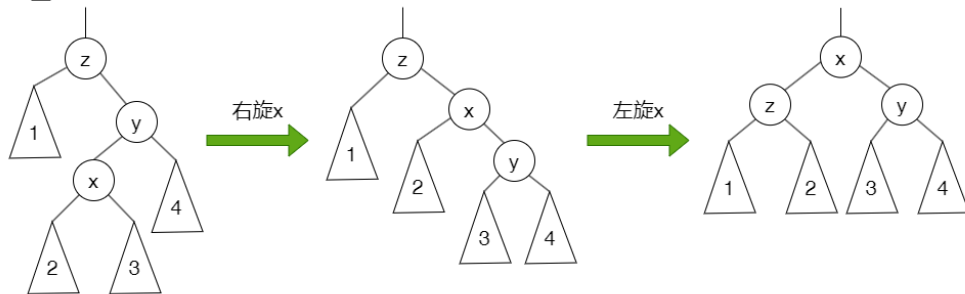
LL型



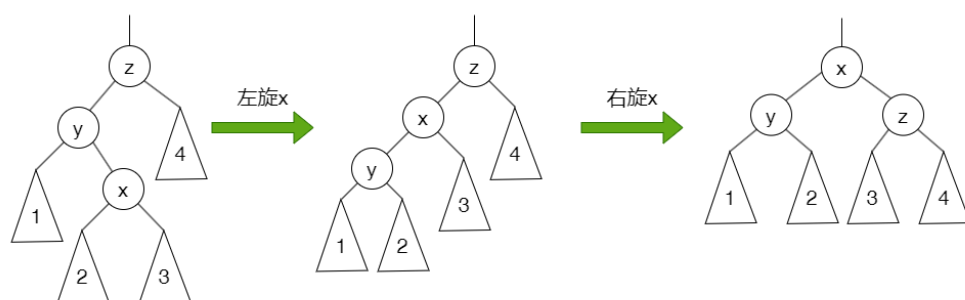
RR型



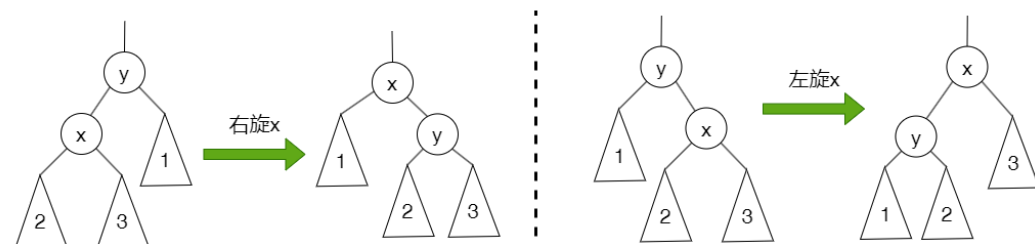
LR型



RL型



L/R型



旋转的规则可以总结成：

- 如果 z 不存在，则只需对 x 旋转一次即可。

- 否则如果 z 存在，且 x, y, z 共线 (x, y 同左或者同右)，则先旋转 y ，再旋转 x 。
- 如果 x, y, z 不共线，则两次旋转 x 。

```
// splay 操作：将结点 x 旋转至 k 下面
// 如果 x 的结点值 < k 的结点值，则操作后 x 是 k 的左子结点
// 如果 x 的结点值 > k 的结点值，则操作后 x 是 k 的右子结点
// 特别地，当 k=nullptr 时，x 会旋转成根
void splay(Node *x, Node *k = nullptr) {
    for(Node *y = x->fa; (y=x->fa) != k; rotate(x)) {
        if(y->fa != k)
            rotate(x->is_rson == y->is_rson ? y : x); // 三点
        // 共线就转 y 否则转 x
    }
    if(k == nullptr) root = x; // 更新根
}
```

插入

插入操作是一个比较复杂的过程，具体步骤如下（假设插入的值为 val ）：

- 如果树空了，则直接插入根并退出。
- 如果当前节点的权值等于 val 则增加当前节点的大小并更新节点和父亲的信息，将当前节点进行 Splay 操作。
- 否则按照二叉查找树的性质向下找，找到空节点就插入即可（请不要忘记 Splay 操作）。

```
void insert(int val) {
    if(root == nullptr) { // 空树，插入新结点
        root = new Node(val);
        return;
    }
    Node *x = root, *y = nullptr;
    while(1) {
        if(x == nullptr) { // 找到空位，插入 val
            x = Node(val);
            x->fa = y;
            y->ch[val > y->val] = x;
            splay(x);
        }
    }
}
```



```

        return;
    }
    if(x->val == val) { // 存在值为 val 的结点，更新结点
        x->cnt++;
        x->siz++;
        splay(x);
        return;
    }
    y = x;
    x = x->ch[val > x->val];
}
}

```

查询 val 的排名

但是，有些时候，*val* 可能不在树中，上面代码会那么此时只能换一种方法了——

- 先查找树中是否存在 *val*，如果存在，则其已经成为树根，否则，插入它，它仍旧会是根。
- 然后我们直接查询根的左子树大小即可，*val* 在树中的排名就是左子树大小 + 1。

```

// 查询 val 在树中有多少个数小于它，val 可以不在树内，注意，返回的不是排名
int query_rank(const int& val) {
    Node *x = find(val);
    bool flag = false; // val 是否是插入的

    if(x == nullptr) { // 如果不存在 val，则
        insert(val);
        flag = true;
    }
    int ans = 0;
    if(root->ch[0] != nullptr) ans += root->ch[0]->siz;
    if(flag) erase(val);
    return ans;
}

```

假设 val 值确保存在于树中，那么求排名的过程也可以这么处理——

根据二叉查找树的定义和性质，显然可以按照以下步骤查询 val 的排名：

- 如果 val 比当前节点的权值小，向其左子树查找。
- 如果 val 比当前节点的权值大，将答案加上左子树 siz 和当前节点 cnt 的大小，向其右子树查找。
- 如果 val 与当前节点的权值相同，将答案加 1 并返回。

注意最后需要进行 Splay 操作。

```
// 查询 val 的排名，若排名不存在，返回 0
int query_rank(const int& val) {
    int res = 0;
    Node *x = root;
    while(1) {
        if(x == nullptr) return 0;
        if(val < x->val) // 在左子树
            x = x->ch[0];
        else {
            res += (x->ch[0]==nullptr ? 0 : x->ch[0]->siz);
            if(val == x->val) {
                splay(x); // 找到了，旋转到根，再返回
                return res + 1;
            }
            // 在右子树上
            res += x->cnt;
            x = x->ch[1];
        }
    }
}
```

查询排名 rank 的数

设 $rank$ 为剩余排名，具体步骤如下：

- 如果左子树非空且剩余排名 $rank$ 不大于左子树的大小 siz ，那么向左子树查找。

- 否则将 $rank$ 减去左子树的和根的大小。如果此时 $rank$ 的值小于等于 0，则返回根节点的权值，否则继续向右子树查找。

```
// 查询排名为 rank 的数，如果排名不存在，返回无穷大
int query_val(const int& rank) {
    Node *x = root;
    while(1) {
        if(x == nullptr) return 0x3f3f3f3f;
        if(x->ch[0] != nullptr && rank <= x->ch[0]->siz)
            x = x->ch[0]; // 排名在左子树内
        else {
            rank -= x->cnt + x->ch[0]->siz;
            if(rank <= 0) { // 当前结点排名 rank
                splay(x);
                return x->val;
            }
            x = x->ch[1]; // 只能在右子树上了
        }
    }
}
```

查询前驱

前驱定义为小于 val 的最大的数，那么查询前驱可以转化为：将 val 插入（此时 val 已经在根的位置了），前驱即为 val 的左子树中最右边的节点，最后将 val 删除即可。

完整的查询前驱需要有 **插入 val** 、**查找左子树的最右边结点**、**删除 val** 这三个过程，这里我们只给出中间的查找左子树的最右边结点的过程，将其封装成函数。

```

// 查询前驱（这里假设已经将 val 旋转至根）结点，如果不存在，返回空
Node* query_pre() {
    Node *x = root->ch[0]; // 根的左子结点
    if(x == nullptr) return x;

    while(x->ch[1] != nullptr) x = x->ch[1];
    splay(x);
    return x;
}

```

完整的封装函数如下：

```

// 查询 val 的前驱结点地址，如果不存在，返回空
Node* query_prev(const int& val) {
    Node *x = find(val);
    bool flag = false;
    if(x==nullptr) {
        insert(val);
        x = root;
        flag = true;
    }
    if(x->ch[0] == nullptr) {
        if(flag) erase(val);
        return nullptr;
    }
    x = x->ch[0];
    while(x->ch[1] != nullptr) x = x->ch[1];
    if(flag) erase(val);
    return x;
}

// 查询 val 的前驱，如果不存在，返回 -1
int query_prev_val(const int& val) {
    Node *x = query_prev(val);
    if(x == nullptr) return -1;
    return x->val;
}

```

查询后继

后继定义为大于 *val* 的最小的数，查询方法和前驱类似：*val* 的右子树中最左边的节点。

```
// 查询后继（这里假设已经将 val 旋转至根），如果不存在，返回空
Node* query_nxt() {
    Node *x = root->ch[1]; // 根的右子结点
    if(x == nullptr) return x;

    while(x->ch[0] != nullptr) x = x->ch[0];
    splay(x);
    return x;
}
```

完整的封装函数如下：

```
// 查询 val 的后继结点地址，如果不存在，返回空
Node* query_next(const int& val) {
    Node *x = find(val);
    bool flag = false;
    if(x==nullptr) {
        insert(val);
        x = root;
        flag = true;
    }
    if(x->ch[1] == nullptr) {
        if(flag) erase(val);
        return nullptr;
    }
    x = x->ch[1];
    while(x->ch[0] != nullptr) x = x->ch[0];
    if(flag) erase(val);
    return x;
}

// 查询 val 的后继，如果不存在，返回 -1
int query_next_val(const int& val) {
    Node *x = query_next(val);
    if(x == nullptr) return -1;
}
```

```
return x->val;  
  
}
```

合并两棵树

合并两棵 Splay 树，设两棵树的根节点分别为 x 和 y ，那么我们要求 x 树中的最大值小于 y 树中的最小值。合并操作如下：

- 如果 x 和 y 其中之一或两者都为空树，直接返回不为空的那一棵树的根节点或空树。
- 否则将 x 树中的最大值 Splay 到根，然后把它的右子树设置为 y 并更新节点的信息，然后返回这个节点。

代码：无，该操作实现在“删除”操作中。

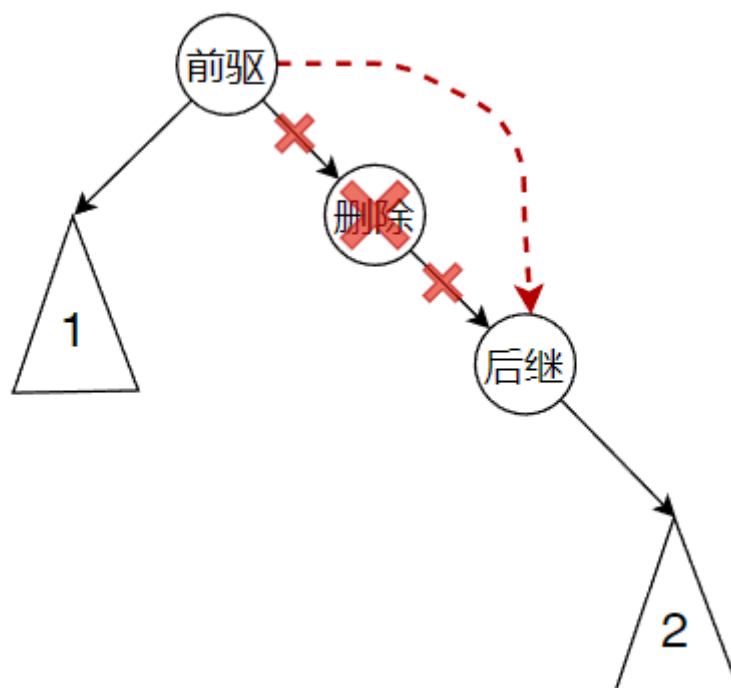
删除

方法一

删除操作也是一个比较复杂的操作，具体步骤如下：

首先将 val 旋转到根的位置。

- 如果 $cnt[x] > 1$ （有不只一个 x ），那么将 $cnt[x]$ 减 1 并退出。
- 否则，合并它的左右两棵子树即可（参加上面的合并两棵树操作）。



```
// 删除 val
void erase(const int& val) {
    Node *x = find(val); // 将 val 旋转到根 x = root
    if(x == nullptr) return;
    if(x->cnt > 1) {
        x->cnt--;
        x->siz--;
        return;
    } // 需要删除根，合并两棵子树
    // 如果左右子树皆空
    if(x->ch[0]==nullptr && x->ch[1]==nullptr) {
        delete x;
        root = nullptr;
        return;
    }
    // 左子树是空的
    if(x->ch[0]==nullptr) {
        root = x->ch[1];
        x->ch[1]->fa = nullptr;
        delete x; // 删除原本的根
        return;
    }
    // 右子树是空的
    if(root->ch[1]==nullptr) {
```

```

    root = x->ch[0];
    x->ch[0]->fa = nullptr;
    delete x; // 删除原本的根
    return;
}

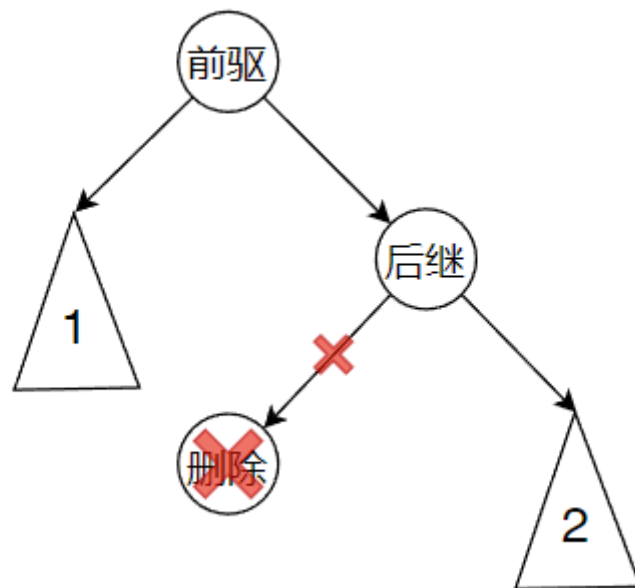
// 左右子树皆不空，将左子树最大值旋到根，然后合并两棵子树
// 这里 x->val 一定存在，因此，进入 query_prev 函数后不会存在
再调用 insert+erase，因此不会存在互相调用的情况
Node *y = query_prev(x->val); // y 是根左子树的最大值，y 必
然无右子结点
splay(y); // 将 y 旋至根，此时 y->ch[1] = x, x->ch[1] = 后
继
root->ch[1] = x->ch[1];
x->ch[1]->fa = root;
delete x; // 删除原本的根
root->update_siz();
}

```

方法二

我们也可以通过巧妙的 Splay 操作来方便地实现删除功能——

- 先找到要删除数 val 的前驱和后继，然后把前驱转到根，后继转到根节点下，也就是前驱的子节点，此时显然有后继是前驱的右儿子。
- 因为 **前驱** $< val <$ **后继**，因此 val 结点一定是后继的左子结点。
- 根据前驱和后继的定义，后继一定**有且仅有一个左儿子**，也就是我们要删除的结点 val 。
- 如果 val 对应 $cnt > 1$ ， $cnt - 1$ 即可，否则就删除该结点。



但，这也只是理想情况，实际问题中，前驱和后继不一定存在，所以情况比较复杂，这也导致代码实现时要进行充分的讨论才可以。

```
void erase(const int &val) {
    Node *x = find(val);

    if (x == nullptr) return;
    if (x->cnt > 1) {
        x->cnt--;
        x->siz--;
        return;
    }

    Node *nxt = query_next(val);
    Node *pre = query_prev(val);

    if (pre == nullptr) {
        root = x->ch[1];
        if (root == nullptr) return;

        root->fa = nullptr;
        delete x;
        return;
    }

    if (nxt == nullptr) {
```

```

        root = x->ch[0];
        root->fa = nullptr;
        delete x;
        return;
    }

    splay(pre, nullptr); // 把前驱转到根
    splay(nxt, pre); // 把后继转到根的右子结点
    x = nxt->ch[0]; // 此时 val 一定是后继的左子结点
    delete x;
    nxt->ch[0] = nullptr;
}

```

完整代码

```

#include <bits/stdc++.h>
using namespace std;
struct Node {
    Node *ch[2], *fa; // 左右孩子指针和父指针
    int val, cnt, siz;
    Node(int val): val(val), cnt(1), siz(1) {
        ch[0] = ch[1] = fa = nullptr;
    }
    bool get() { // 判断当前结点是不是右孩子, 用于 splay 操作
        // 如果是右孩子, 返回 1, 否则左孩子, 返回 0
        return fa->ch[1] == this;
    }
    void update_siz() { // 更新结点信息
        siz = cnt;
        if(ch[0] != nullptr) siz += ch[0]->siz;
        if(ch[1] != nullptr) siz += ch[1]->siz;
    }
};

class Splay{
private:
    Node *root = nullptr;
    void rotate(Node *x) {

```

```

// y: 父结点   k: x 是否是右孩子
Node *y = x->fa;
int k = x->get(); // 当前结点 x 在父结点的 ch[] 中的下标
// x 上旋 (左旋 or 右旋)
y->ch[k] = x->ch[k^1];
if(x->ch[k^1] != nullptr)
    x->ch[k^1]->fa = y; // 将 x 的反向孩子链向 y
if((x->fa = y->fa) != nullptr) // 将 x 上旋至 y 的位置
    x->fa->ch[y->get()] = x;
y->fa = x; // y 下旋成 x 子结点
x->ch[k^1] = y;
y->update_siz();
x->update_siz();
// 更新 root
if(x->fa == nullptr) root = x;
}

// splay 操作: 将结点 x 旋转至 k 下面
// 如果 x 的结点值 < k 的结点值, 则操作后 x 是 k 的左子结点
// 如果 x 的结点值 > k 的结点值, 则操作后 x 是 k 的右子结点
// 特别地, 当 k=nullptr 时, x 会旋转成根
void splay(Node *x, Node *k = nullptr) {
    if(x == nullptr) return;
    for(Node *y = x->fa; (y=x->fa) != k; rotate(x)) {
        if(y->fa != k)
            rotate(x->get() == y->get() ? y : x);
        // 三点共线就转 y 否则转 x
    }
}

// 查询 val 对应的结点
Node* find(const int& val) {
    Node *x = root;
    while(x != nullptr && x->val != val)
        x = x->ch[val > x->val]; // 往对应子树找
    if(x != nullptr) // 如果树中确实存在 val, 则 splay
        splay(x); // 将 val 旋转到根
    return x;
}

// 查询 val 的前驱结点地址, 如果不存在, 返回空
Node* query_prev(const int& val) {

```

```

Node *x = find(val);
bool flag = false;
if(x==nullptr) {
    insert(val);
    x = root;
    flag = true;
}
if(x->ch[0] == nullptr) {
    if(flag) erase(val);
    return nullptr;
}
x = x->ch[0];
while(x->ch[1] != nullptr) x = x->ch[1];
if(flag) erase(val);
return x;
}

// 查询 val 的后继结点地址，如果不存在，返回空
Node* query_next(const int& val) {
    Node *x = find(val);
    bool flag = false;
    if(x==nullptr) {
        insert(val);
        x = root;
        flag = true;
    }
    if(x->ch[1] == nullptr) {
        if(flag) erase(val);
        return nullptr;
    }
    x = x->ch[1];
    while(x->ch[0] != nullptr) x = x->ch[0];
    if(flag) erase(val);
    return x;
}

public:
    // 插入
    void insert(const int& val) {
        if(root == nullptr) { // 空树，插入新结点
            root = new Node(val);

```

```

        return;
    }
    Node *x = root, *y = nullptr;
    while(1) {
        if(x == nullptr) { // 找到空位, 插入 val
            x = new Node(val);
            x->fa = y;
            y->ch[val > y->val] = x;
            splay(x);
            return;
        }
        if(x->val == val) { // 存在值为 val 的结点, 更新结点
            x->cnt++;
            x->siz++; // 因为后面会 splay 可以不写
            splay(x);
            return;
        }
        y = x; // 继续往下找
        x = x->ch[val > x->val];
    }
}

////////////////////////////////////
// 删除 val
void erase(const int& val) {
    Node *x = find(val); // 将 val 旋转到根 x = root
    if(x == nullptr) return;
    if(x->cnt > 1) {
        x->cnt--;
        x->siz--;
        return;
    } // 需要删除根, 合并两棵子树
    // 如果左右子树皆空
    if(x->ch[0]==nullptr && x->ch[1]==nullptr) {
        delete x;
        root = nullptr;
        return;
    }
    // 左子树是空的

```

```

    if(x->ch[0]==nullptr) {
        root = x->ch[1];
        x->ch[1]->fa = nullptr;
        delete x; // 删除原本的根
        return;
    }
    // 右子树是空的
    if(root->ch[1]==nullptr) {
        root = x->ch[0];
        x->ch[0]->fa = nullptr;
        delete x; // 删除原本的根
        return;
    }
    // 左右子树皆不空，将左子树最大值旋到根，然后合并两棵子树
    // 这里 x->val 一定存在，因此，进入 query_prev 函数后不会
    存在再调用 insert+erase，因此不会存在互相调用的情况
    Node *y = query_prev(x->val); // y 是根左子树的最大值，y
    必然无右子结点
    splay(y); // 将 y 旋至根，此时 y->ch[1] = x, x->ch[1]
    = 后继
    root->ch[1] = x->ch[1];
    x->ch[1]->fa = root;
    delete x; // 删除原本的根
    root->update_siz();
}
// 如果确保 val 一定存在，也可以这么实现
void _erase(const int &val) {
    Node *x = find(val);

    if (x == nullptr)
        return;

    if (x->cnt > 1) {
        x->cnt--;
        x->siz--;
        return;
    }

    Node *nxt = query_next(val);

```

```

Node *pre = query_prev(val);

if (pre == nullptr) {
    root = x->ch[1];

    if (root == nullptr)
        return;

    root->fa = nullptr;
    delete x;
    return;
}

if (nxt == nullptr) {
    root = x->ch[0];
    root->fa = nullptr;
    delete x;
    return;
}

splay(pre, nullptr); // 把前驱转到根
splay(nxt, pre); // 把后继转到根的右子结点
x = nxt->ch[0]; // 此时 val 一定是后继的左子结点
delete x;
nxt->ch[0] = nullptr;
}

// 查询排名为 rank 的数
int query_val(int rank) {
    Node *x = root;
    while(1) {
        //if(x == nullptr) return 0x3f3f3f3f;
        if(x->ch[0] != nullptr && rank <= x->ch[0]->siz)
            x = x->ch[0];
        else {
            rank -= x->cnt + (x->ch[0]==nullptr?0:x-
>ch[0]->siz);

            if(rank <= 0) { // 当前结点排名 rank
                splay(x);
                return x->val;
            }
        }
    }
}

```

```

        }
        x = x->ch[1];
    }
}

// 查询 val 在树中有多少个数小于它, val 可以不在树内
int query_rank(const int& val) {
    Node *x = find(val);
    bool flag = false; // val 是否是插入的

    if(x == nullptr) { // 如果不存在 val, 则
        insert(val);
        flag = true;
    }
    int ans = 0;
    if(root->ch[0] != nullptr) ans += root->ch[0]->siz;
    if(flag) erase(val);
    return ans;
}

// 查询 val 的前驱, 如果不存在, 返回 -1
int query_prev_val(const int& val) {
    Node *x = query_prev(val);
    if(x == nullptr) return -1;
    return x->val;
}

// 查询 val 的后继, 如果不存在, 返回 -1
int query_next_val(const int& val) {
    Node *x = query_next(val);
    if(x == nullptr) return -1;
    return x->val;
}

void dfs(Node *x) {
    if(x==nullptr) return;
    dfs(x->ch[0]);
    printf("dfs %d %d %p %p\n", x->val, x->siz, x->ch[0], x->ch[1]);
    dfs(x->ch[1]);
}

```



```

};

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n, op, x;
    cin >> n;
    Splay tr;
    while(n--) {
        cin >> op >> x;
        if(op == 0) tr.insert(x);
        if(op == 1) tr.erase(x);
        if(op == 2) cout << tr.query_val(x) << '\n';
        if(op == 3) cout << tr.query_rank(x) << '\n';
        if(op == 4) cout << tr.query_prev_val(x) << '\n';
        if(op == 5) cout << tr.query_next_val(x) << '\n';
    }
    return 0;
}

```

Splay 维护序列（区间）

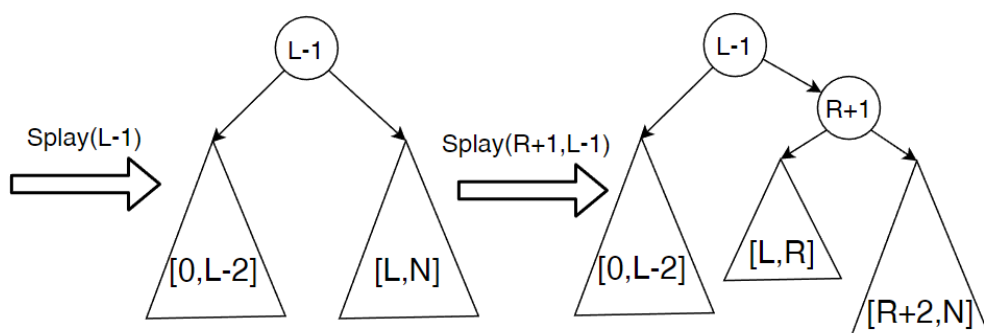
Splay 也可以运用在序列上，用于维护区间信息。与线段树对比，Splay 常数较大，但是支持更复杂的序列操作，如区间翻转等。

将序列建成的 Splay 有如下性质：

- Splay 的中序遍历相当于原序列从左到右的遍历。
- Splay 上的一个节点代表原序列的一个元素；Splay 上的一颗子树，代表原序列的一段区间。

因为有 splay 操作，可以快速提取出代表某个区间的 Splay 子树——如果我们想操作区间 $[L, R]$ ，那么可以通过如下操作将区间 $[L, R]$ 提取出来：

- 将代表 a_{L-1} 的节点 splay 到根，此时右子树为区间 $[L, N]$
- 再将代表 a_{R+1} 的节点 splay 到根的右儿子，此时 a_{R+1} 的节点的左子树中序遍历为序列 $a[L, R]$ ，故其为区间 $[L, R]$ 代表的子树。



一般会建立左右两个哨兵节点 0 和 $n + 1$ ，放在数列的最开头和最结尾，防止 $L - 1$ 或 $R + 1$ 超出数列范围。

在操作之前，你需要先把这颗 Splay 建出来。根据 Splay 的特性，直接建出一颗只有右儿子的链即可，时间复杂度仍然是正确的。

区间翻转

和分裂合并的 *FHQ-Treap* 是一样的，使用懒标记的技巧即可。

先将询问区间的子树提取出来。因为是区间翻转，我们需要将这棵子树的中序遍历顺序翻转。

一个暴力做法是每次将根节点的左右儿子交换，然后递归左右子树做同样的操作，这样复杂度为 $O(n)$ ，不可承受。可以考虑使用懒标记，先给根打上「翻转标记」并交换其左右儿子。当递归到一个带懒标记的点时，将懒标记下传即可。

```
// 翻转区间
void reverse_tag(Node *x) {
    if(x == nullptr) return;
    swap(x->ch[0], x->ch[1]);
    tag ^= 1;
}

// 懒标记下放
void pushdown(Node *x) {
    if(x == nullptr || !tag) return;
    reverse_tag(x->ch[0]);
    reverse_tag(x->ch[1]);
    tag = 0;
}
```

完整代码

平衡树翻转区间

```
#include <bits/stdc++.h>
using namespace std;
int n, m;
struct Node {
    Node *ch[2], *fa; // 左右孩子指针和父指针
    int val, siz;
    bool tag;
    Node(int val): val(val), siz(1) {
        tag = false;
        ch[0] = ch[1] = fa = nullptr;
    }
    bool get() { // 判断当前结点是不是右孩子，用于 splay 操作
        return fa->ch[1] == this;
    }
    void push_up() {
        siz = 1;
        if(ch[0] != nullptr) siz += ch[0]->siz;
        if(ch[1] != nullptr) siz += ch[1]->siz;
    }
    void push_down() {
        if(!tag) return;
        swap(ch[0], ch[1]);
        if(ch[0] != nullptr) ch[0]->tag ^= 1;
        if(ch[1] != nullptr) ch[1]->tag ^= 1;
        tag = 0;
    }
};

class Splay{
private:
    void rotate(Node *x) {
        Node *y = x->fa; // y: 父结点 k: x 是否是右孩子
        int k = x->get(); // 当前结点 x 在父结点的 ch[] 中的下标
        // x 上旋 (左旋 or 右旋)
        y->ch[k] = x->ch[k^1];
```

```

        if(x->ch[k^1] != nullptr)
            x->ch[k^1]->fa = y; // 将 x 的反向孩子链向 y
        if((x->fa = y->fa) != nullptr) // 将 x 上旋至 y 的位置
            x->fa->ch[y->get()] = x;
        y->fa = x; // y 下旋成 x 子结点
        x->ch[k^1] = y;
        y->push_up();
        x->push_up();
        // 更新 root
        if(x->fa == nullptr) root = x;
    }
}

void splay(Node *x, Node *k = nullptr) {
    if(x == nullptr) return;
    for(Node *y = x->fa; (y=x->fa) != k; rotate(x)) {
        if(y->fa != k)
            rotate(x->get() == y->get() ? y : x);
        // 三点共线就转 y 否则转 x
    }
}

public:
    Node *root = nullptr;
    // 插入
    void insert(const int& val) {
        if(root == nullptr) { // 空树, 插入新结点
            root = new Node(val);
            return;
        }
        Node *x = root, *y = nullptr;
        while(1) {
            if(x == nullptr) { // 找到空位, 插入 val
                x = new Node(val);
                x->fa = y;
                y->ch[val > y->val] = x;
                splay(x);
                return;
            }
            x->push_down(); // 这里可以不写
            y = x; // 继续往下找(值不会出现重复)
            x = x->ch[val > x->val];
        }
    }
}

```

```

    }
}
// 查询排名为 rank 的结点, 需要用 siz
Node* query_kth(int rank) {
    Node *x = root;
    while(1) {
        x->push_down();
        if(x->ch[0] != nullptr && rank <= x->ch[0]->siz)
            x = x->ch[0];
        else {
            rank -= 1 + (x->ch[0]==nullptr?0:x->ch[0]-
>siz);

            if(rank <= 0) { // 当前结点排名 rank
                splay(x);
                return x;
            }
            x = x->ch[1];
        }
    }
}

// 翻转区间
void reverse(int l, int r) {
    Node *y = query_kth(r+2), *x = query_kth(l); // 此时 x
是根
    splay(x);
    splay(y, x);
    y->ch[0]->tag ^= 1;
}

void dfs(Node *x) {
    if(x==nullptr) return;
    x->push_down();
    dfs(x->ch[0]);
    if(x->val > 0 && x->val < n+1) cout << x->val << ' ';
    dfs(x->ch[1]);
}

void clear(Node *x) { // 递归释放结点
    if(x==nullptr) return;
    x->push_down();
    clear(x->ch[0]);
}

```

```

        clear(x->ch[1]);
        delete x;
        x = nullptr;
    }
};

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int l, r;
    cin >> n >> m;
    Splay tr;
    for(int i = 0; i <= n+1; i++){
        tr.insert(i);
    }
    while(m--){
        cin >> l >> r;
        tr.reverse(l, r);
    }
    tr.dfs(tr.root);
    tr.clear(tr.root); // 对于 new 创建的结点，要 delete 掉，否则会发生内存泄漏
    return 0;
}

```

蓝桥杯真题

① Note

蓝桥杯下面的平衡树真题较难，可以当作冲刺国特的练习题。

圆圈舞（蓝桥杯C/C++2016A组国赛）

习题

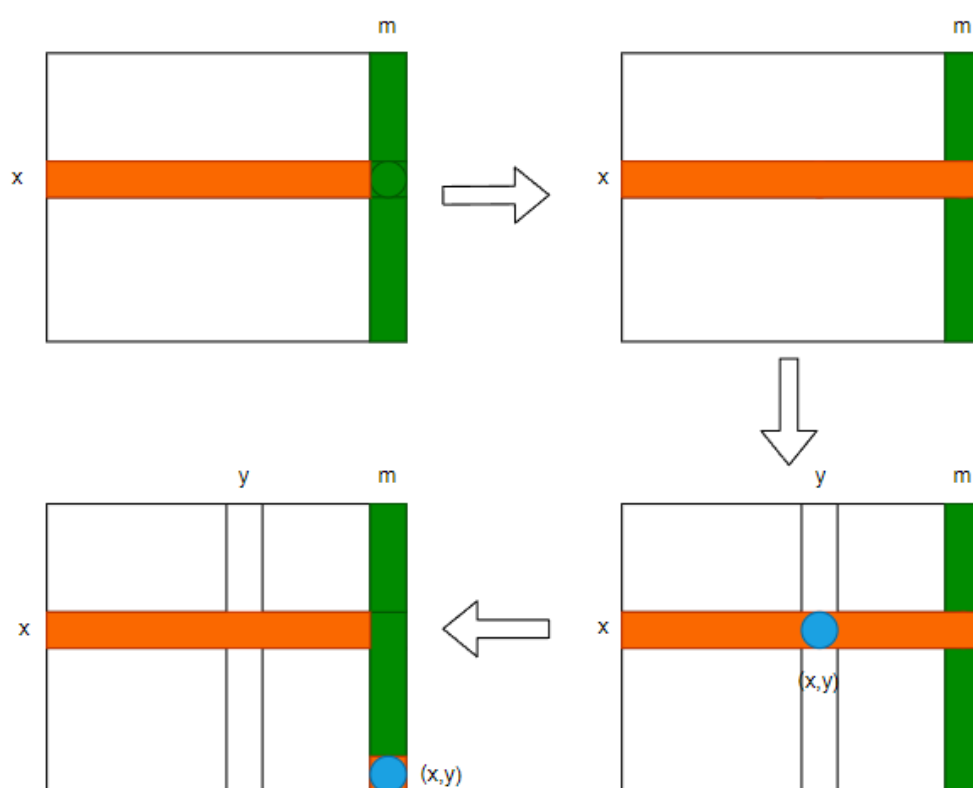
列队

如果把所有结点都建出来，那么空间都受不了，得益于离队事件只有 q 次，所以真正会有大变化的点不多。我们来仔细分析一下每次离队会带来什么。

每次当 (x, y) 位置的人离队时，先是对于行，从第 y 个位置开始，之后每个位置都会往左移，往前移会导致最后一列空一个位置，最后一列的所有点又会往前移。自此，我们发现了最后一列的重要性，最后一列非常重要，所以我们**单独**维护。

因为涉及到加点，删点，所以考虑用平衡树维护最后一列。每次 (x, y) 出列，就把平衡树中第 x 个数提出来，再在平衡树的最后边加入一个原来 (x, y) 代表的编号。

接着考虑除了最后一列，其他列怎么维护，如果也用平衡树把所有点提出来非常不现实。但我们发现离队事件并不是很多，这意味着大部分的学生都是循规蹈矩地向左移动一小格，对于这些循规蹈矩的学生，我们就不再理会他们了。所以我们还是可以每一行建一个平衡树，用来存那些编号不是那么有规律的学生，当代表最后一列的平衡树中，第 x 个学生被移出时，就在代表第 x 行的平衡树的末尾加入这个学生。然后我们再在第 x 行的平衡树中找到排名是 y 的学生，将其移出，加入最后一列的最后。



在维护每一行的平衡树时，我们发现有些人从始至终都是左右相邻的，这些人对应的 splay 节点可以合并。

于是我们在维护 splay 的时候，所有连续的数的区间我们用一个节点维护，记录这个节点表示的区间的左右端点；

如果我们发现要弹出的数在某个节点内部，我们就把它拆开，拆成3个节点（其中中间那个节点只有一个数，就是我们要的那个数），拆点的过程可以直接跑splay上的insert，删除中间那个节点就是splay上的删除。

如此，我们要实现的平衡树 splay 需要满足：

1. 每个结点维护的是一段连续的学生编号 $[l, r)$ ，对于最后一列，每个结点维护 $[l, l + 1)$ 。
2. 支持查询第 k 个学生的编号，并删除。
3. 支持在树的末尾插入一个新的学生结点。
4. 支持将一个结点 $[l, r)$ 分裂成 $[l, mid)$ 和 $[mid, r)$ ，以方便查询第 k 大结点并删除。

所以，我们无需实现 Splay 的所有功能。参考代码如下：

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 3e5+5;
int n, m, q;
struct Node {
    Node *ch[2], *fa; // 左右孩子指针和父指针
    ll siz, l, r; // 结点代表区间 [l,r)，左闭右开
    Node(ll l, ll r):l(l), r(r) {
        siz = r-l;
        ch[0] = ch[1] = fa = nullptr;
    }
    bool get() { // 判断当前结点是不是右孩子，用于 splay 操作
        return fa->ch[1] == this;
    }
    void push_up() { // 更新节点信息
        siz = r-l;
        if(ch[0] != nullptr) siz += ch[0]->siz;
        if(ch[1] != nullptr) siz += ch[1]->siz;
    }
};
```



```

    }
};

class Splay{
private:
    void rotate(Node *x) {
        Node *y = x->fa;    // y: 父结点    k: x 是否是右孩子
        int k = x->get();    // 当前结点 x 在父结点的 ch[] 中的下标
        // x 上旋 (左旋 or 右旋)
        y->ch[k] = x->ch[k^1];
        if(x->ch[k^1] != nullptr)
            x->ch[k^1]->fa = y; // 将 x 的反向孩子链向 y
        if((x->fa = y->fa) != nullptr) // 将 x 上旋至 y 的位置
            x->fa->ch[y->get()] = x;
        y->fa = x;    // y 下旋成 x 子结点
        x->ch[k^1] = y;
        y->push_up();
        x->push_up();
        // 更新 root
        if(x->fa == nullptr) root = x;
    }

    void splay(Node *x, Node *k = nullptr) {
        if(x == nullptr) return;
        for(Node *y = x->fa; (y=x->fa) != k; rotate(x)) {
            if(y->fa != k)
                rotate(x->get() == y->get() ? y : x);
            // 三点共线就转 y 否则转 x
        }
    }

    // 将结点 x:[l,r) 分裂成 [l,l+k) 和 [l+k, r)
    // 分裂后 x 的区间的前 k 个数还在 x 结点上, 返回分裂后的根
    Node* split_node(Node *x, ll k) {
        ll mid = x->l + k;    // [l, mid) + [mid, r)
        Node *y = new Node(mid, x->r);
        x->r = mid;
        // 把 y 插入 x 右子树
        if(x->ch[1] == nullptr) {
            x->ch[1] = y;
            y->fa = x;
        } else {

```

```

        Node *t = x->ch[1];
        while(t->ch[0] != nullptr) t = t->ch[0];
        t->ch[0] = y;
        y->fa = t;
        while(t != x) {
            t->push_up();
            t = t->fa;
        }
    }
    splay(y);
    return y;
}

public:
    Node *root = nullptr;
    // 弹出第 k 大的数
    ll pop_kth(ll k) {
        ll q = k;
        Node *x = root;
        while(1) {
            if(x->ch[0] != nullptr && x->ch[0]->siz >= k)
                x = x->ch[0];
            else {
                k -= (x->ch[0]==nullptr?0:x->ch[0]->siz);
                if(k <= x->r - x->l) {
                    if(k != x->r-x->l) split_node(x, k);
                    if(k != 1) x = split_node(x, k-1);
                    break;
                } else {
                    k -= x->r - x->l;
                    x = x->ch[1];
                }
            }
        }
    }
    // 删除 x
    ll kth = x->l;
    splay(x);
    if(x->ch[0] == nullptr) { // 左子树为空
        root = x->ch[1];
        if(root != nullptr) root->fa = nullptr;
    }
}

```

```

    } else {
        Node *t = x->ch[0];
        t->fa = nullptr;
        while(t->ch[1] != nullptr) t = t->ch[1];
        splay(t); // 将 t 旋至左子结点的根
        t->ch[1] = x->ch[1];
        root = t;
        if(x->ch[1] != nullptr) x->ch[1]->fa = t;
        root->push_up();
    }
    delete x;
    return kth;
}

// 在最后插入结点 k
void push_back(ll k) {
    Node *x = new Node(k, k+1);
    if(root == nullptr) root = x;
    else {
        Node *y = root;
        while(y->ch[1] != nullptr) y = y->ch[1];
        splay(y);
        y->ch[1] = x;
        x->fa = y;
        y->push_up();
    }
}

// 递归释放结点
void dfs_del(Node *x) {
    if(x==nullptr) return;
    dfs_del(x->ch[0]); dfs_del(x->ch[1]);
    delete x;
    x = nullptr;
}

};

Splay tr[N];
int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m >> q;

```

```

// 维护每一行 (的 1~m-1 列)
for(ll i = 1; i <= n; i++) tr[i].init((i-1)*m+1, i*m); //
[(i-1)*m+1, i*m)
// tr[0]: 单独维护每一行的最后一列
for(ll i = 1; i <= n; i++) tr[0].push_back(i*m);
ll x, y;
while(q--) {
    cin >> x >> y;
    ll tmp = tr[0].pop_kth(x); // 将第 x 行第 m 列的值删除
    tr[x].push_back(tmp); // 该值进入第 x 行末尾
    ll t = tr[x].pop_kth(y); // 删除 (x,y)
    tr[0].push_back(t); // 压入末尾
    cout << t << "\n";
}
// 释放申请的空间
for(int i = 0; i <= n; i++) tr[i].dfs_del(tr[i].root);
return 0;
}

```